NLP UNIT-III

- **Subject**: Artificial Intelligence/NLP
  Presented by: Duggirala Sri Ramya Krishna

  Department: CSE
- College:-Sir C R Reddy Engineering College
- Designation:- Asst professor
- Mail:ramya999mr@gmail.com

Context-Free Grammar (CFG)

A Context-Free Grammar (CFG) is : A set of rules used to describe the structure of sentences.

CFG tells:

- How words combine
- Which sentence forms are valid

<u>Why CFG is important?</u>

- Used in:
  - ✓ Parsing
  - ✓ Compilers
  - ✓ NLP
  - ✓ Understanding grammer

Duggirala Sri Ramya Krishna

Parts of CFG

- CFG consists of four parts:

Non-terminals

Sentence parts like: S, NP, VP

Terminals

Actual words: boy, eats, apple

Start symbol

Usually: S (Sentence)

Production rules

Rules to replace symbols.

Rule format : $A \rightarrow B$

(Replace A by B)

Duggirala Sri Ramya Krishna

LEXICAL RULES (Word-Level Rules)

Category	Rule	Example
Determiner	Det → a, an, the	the bus
Noun	N → boy, apple	teacher
Verb	V → eats, runs	plays
Adjective	Adj → big, red	red flower
Adverb	Adv → quickly, well	ran quickly
Preposition	Prep → in, on, to	on table
Pronoun	Pron → he, she, they	she sings
Auxiliary	Aux → is, are, have	is running

SENTENCE & NOUN PHRASE RULES

Rule	Meaning	Example
S → NP VP	Sentence = subject + predicate	The boy eats
NP → Det N	Article + noun	the dog
NP → Det Adj N	Adjective + noun	the small boy
NP → N	Single noun	apples
NP → Pronoun	Pronoun as NP	she

Duggirala Sri Ramya Krishna

VERB & OTHER PHRASE RULES

Rule	Meaning	Example
VP → V NP	Verb + object	eats rice
VP → V	Verb alone	birds fly
VP → Aux V	Helping + verb	is running
VP → Aux V NP	Helping + verb + object	has bought book
PP → Prep NP	Preposition phrase	in the room
ADJP → Adj	Adjective phrase	happy
ADVP → Adv	Adverb phrase	slowly

Duggirala Sri Ramya Krishna

Example of Context-Free Grammar (CFG)

- **Sentence:** The boy eats apple

- **Grammar Rules:**

- S → NP VP
  NP → Det N
  VP → V NP
  Det → the
  N → boy | apple
  V → eats

How rules are applied

Step 1:

- Start with the start symbol:
- S

Step 2:

- Apply rule S → NP VP
- S becomes:
  NP VP

Step 3:

- Apply rule NP → Det N
- NP becomes:
  Det N
- Now sentence form is:
- Det N VP

Step 4:

- Apply rule VP → V NP
- Becomes:
- Det N V NP

Replacing with Actual Words

Step 5:

- Replace Det → the
- Becomes:
  the N V NP

Step 6:

- Replace N → boy
- Becomes:
  the boy V NP

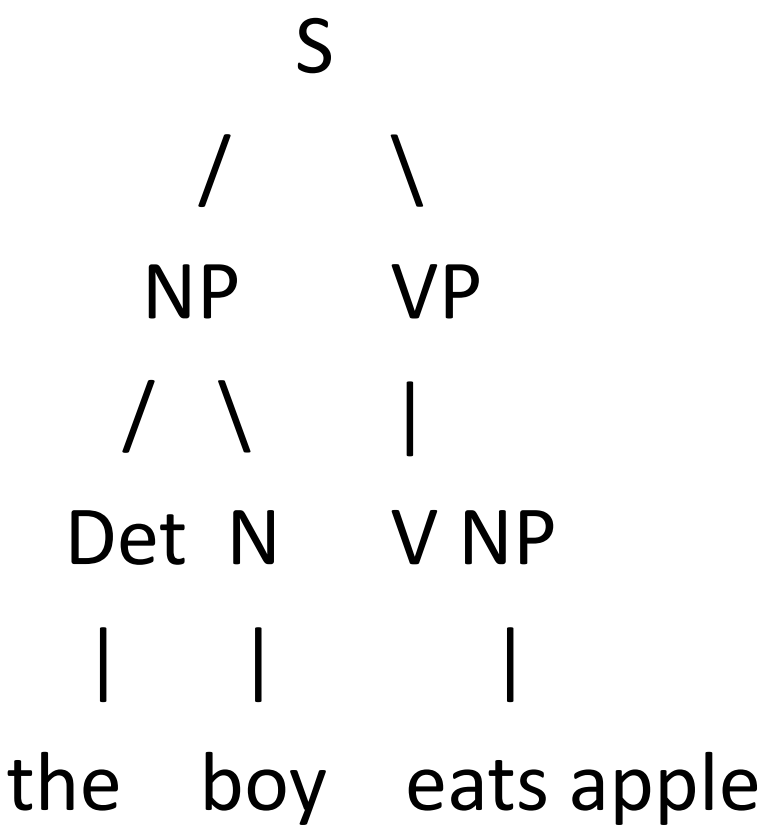
Step 7:

- Replace V → eats
- Becomes:
  the boy eats NP

Step 8:

- Replace NP → N → apple
- Final sentence:
- ✅ **the boy eats apple**

Final structure view:



- CFG shows **how each word fits in the sentence.**

What is a Treebank?

- A Treebank is:A database of sentences where each sentence is stored with its grammar structure (parse tree).

A Treebank is a collection of:

- Sentences
- POS tags
- Parse trees
- Phrase labels

Without Treebank:

- Computers do not know sentence structure.

With Treebank:

- Computers learn grammar from examples.

Duggirala Sri Ramya Krishna

TYPES OF TREEBANKS

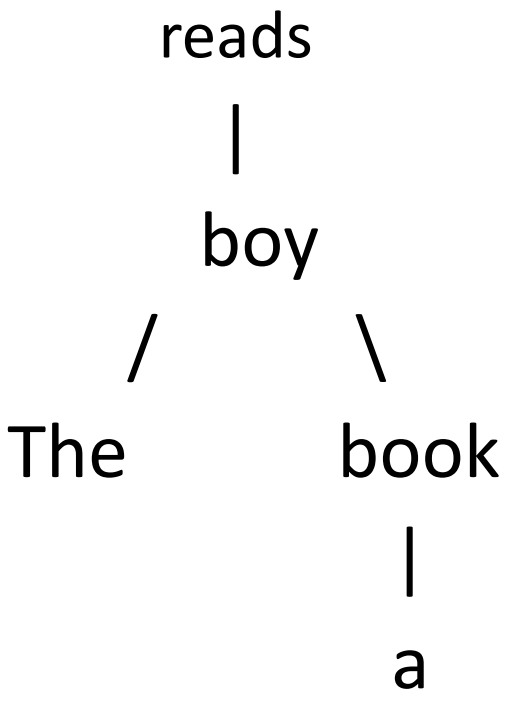
1.Phrase Structure Treebank (Constituency Treebank)

- Shows sentence structure as **phrases inside phrases**.
- **Example Sentence:**
- The boy reads a book
- Ex : refer above cfg example

2.Dependency Treebank:

Shows **word-to-word relationships** instead of phrases.

- **Example Sentence:** The boy reads a book



- **Used in:** Understanding grammatical relations.

3.PropBank / Role-labeled Treebank

"Shows **who does what to whom**."

- **Example:** Ram eats apple

Role	Word
Agent	Ram
Action	eats
Object	apple

4.Parallel Treebank

Contains sentences in **multiple languages with trees**.

English	Hindi
He eats apple	वह सेब खाता है

Both sentences have parse trees.

What Are Normal Forms?

- Normal Forms are Standard, restricted formats for writing grammar rules so that parsing becomes simple and consistent.

Why Normal Forms?

- Reduce grammar complexity
- Make parsing algorithms easier
- Allow automatic processing (like CYK parser)
- Convert long rules into simple two-part rules

Two Important Normal Forms

- Chomsky Normal Form (CNF)
- Greibach Normal Form (GNF)

Duggirala Sri Ramya Krishna

Chomsky Normal Form (CNF)

- CNF Rule Types (ONLY these two allowed)

Rule 1: A → BC

- A, B, C are non-terminals
- Exactly two symbols on the right
- No terminals allowed here

Example:

- S → NP VP
- This is already in CNF because NP and VP are non-terminals.

Rule 2: A → a

- Right side must contain exactly one terminal
- No other symbols

Example:

1.N → boy

2.V → eats

- This is CNF because "boy" and "eats" are terminals.

Converting to CNF

Original Rule (Not CNF): NP → Det Adj N

- This is **not CNF** because:
- It has **three symbols** on the right side.

Step-by-step Conversion to CNF

Step 1: Break into binary rules

- NP → Det X

Step 2: Define X

- X → Adj N

Now BOTH rules fit CNF:

- NP → Det X (A → BC)
- X → Adj N (A → BC)

Final result:

- NP → Det X
- X → Adj N

Greibach Normal Form (GNF)

GNF Rule Format: $A \rightarrow a\ B1\ B2\ \ldots$

Where:

- a = terminal word
- B1, B2… = zero or more non-terminals
- MUST start with a terminal

Meaning

- Leftmost symbol produced is always a word
- Guarantees **left-to-right sentence generation**
- Useful for designing **top-down parsers**

Greibach Normal Form (GNF) – Conversion Example

S → NP VP
NP → Det N
VP → V
Det → the
N → boy
V → runs

- Convert NP to start with a terminal
- Original: NP → Det N

  Det → the
- Replace Det with its terminal: NP → the N

Convert VP to start with terminal

- Original: VP → V

  V → runs
- Replace V with terminal: VP → runs

Convert S to start with terminal

- Original: S → NP VP
- Replace NP with terminal form (the N): S → the N VP
- S → the N VP
- NP → the N
- VP → runs
- Final Grammar in GNF:

S → the N VP
NP → the N
VP → runs

S → NP VP
NP → Det N
VP → V
Det → the
N → boy
V → runs

S → the N VP
NP → the N
VP → runs

Dependency Grammar in NLP

Dependency grammar focuses on:

- Relationships between words in a sentence rather than phrase structure.
- Instead of:
- Building phrases (NP, VP)
  It builds:
- Word-to-word links

Each relation shows:

- Which word depends on which

Duggirala Sri Ramya Krishna

Basic Idea

Head and Dependent

- Every relationship has:

Head word → main word

Dependent word → describes the head

Example:

- Sentence:
- The boy eats apple
- Main verb: eats (HEAD)

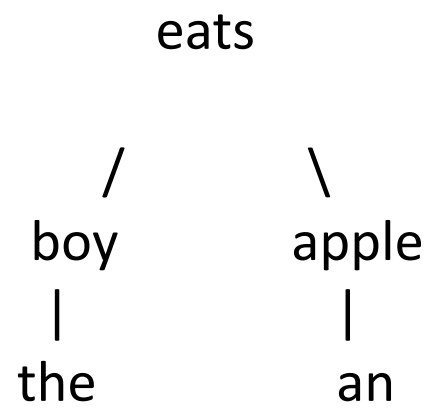
Relations:

- boy → subject of eats
- apple → object of eats
- the → determiner of boy

Simple Dependency Tree Example

Sentence: The boy eats an apple.

Structure:



This tree shows:

- eats = root
- boy = subject
- apple = object
- the, an = determiners

Dependency Relations in NLP

A dependency relation shows:

- How one word depends on another word.
- Each relation has:
- **Head** → main word
- **Dependent** → related word
- **Relation name** → type of link
- Example:
- The boy eats an apple.

<u>Relations:</u>

- boy → eats (subject)
- apple → eats (object)
- the → boy (article)
- an → apple (article)

Duggirala Sri Ramya Krishna

Subject & Object Relations

1. nsubj (Nominal Subject)

- Shows the subject of a verb.
- Example:
- She sings
  sings ← she (**nsubj**)

2. dobj / obj (Direct Object)

- Shows the object of the verb.
- Example:
- He reads a book
  reads ← book (**dobj**)

3. iobj (Indirect Object)

- Shows the receiver of action.
- Example:
- She gave me a pen
  gave ← me (**iobj**)

Modifier Relations

4. amod (Adjectival Modifier)

- Adjective modifies noun.
- Example:
- red apple
  apple ← red (**amod**)

5. advmod (Adverbial Modifier)

- Adverb modifies verb.
- Example:
- runs fast
  runs ← fast (**advmod**)

6. det (Determiner)

- Articles point to nouns.
- Example:
- the boy
  boy ← the (**det**)

Verb & Auxiliary Relations

7. aux (Auxiliary)

- Helping verbs.
- Example:
- is running
  running ← is (**aux**)

8. cop (Copula)

- Linking verb (is, was).
- Example:
- He is a teacher
  teacher ← is (**cop**)

9. neg (Negation)

- Negative words.
- Example:
- She does not like
  like ← not (**neg**)

Prepositional & Case Relations

10. case

- Prepositions that introduce noun phrase.
- Example:
- in the park
  park ← in (**case**)

11. nmod **(Noun Modifier)**

- Noun connected through preposition.
- Example:
- man in office
  man ← office (**nmod**)

Coordination Relations

12. cc (Coordinating Conjunction)

- Example:
- Ram and Shyam
  Shyam ← and (**cc**)

13. conj (Conjunct)

- Words joined by conjunction.
- Example:
- Ram and Shyam
  Ram ← Shyam (**conj**)

Abbrev	Full Form	Meaning	Example
root	Root	Main verb of the sentence	*eats* in "He eats food"
nsubj	Nominal Subject	Subject of verb	*boy* → eats
csubj	Clausal Subject	Clause as subject	*That he came* surprised
obj / dobj	Direct Object	Object of verb	*book* → reads
iobj	Indirect Object	Receiver of action	*me* → gave
obl	Oblique Nominal	Object of preposition	*school* "go to school"
det	Determiner	Article / demonstrative	*the* → boy
amod	Adjectival Modifier	Adjective for noun	*red* → apple
advmod	Adverbial Modifier	Adverb for verb/adjective	*fast* → runs
aux	Auxiliary	Helping verb	*is* → running
cop	Copula	Linking verb	*is* → teacher

Syntactic Parsing (Syntax Analysis) in NLP

Syntactic parsing :

The process of analyzing the **grammatical structure** of a sentence.

It tells:

- Which word is subject
- Which is verb
- Which phrase depends on which
- Whether the sentence is grammatically correct

Simple Example:

Sentence: *The boy eats an apple.*

Parsing identifies:

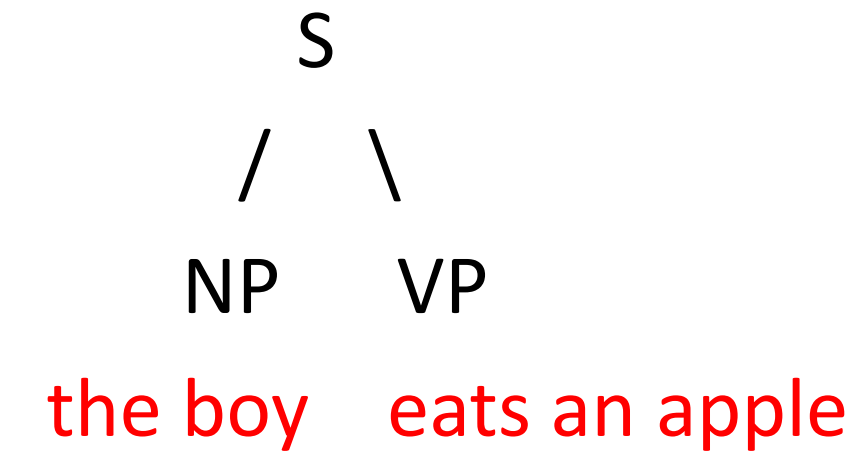
- NP (Noun Phrase): *The boy*
- VP (Verb Phrase): *eats an apple*

Types of Syntactic Parsing

1.Constituency Parsing (Phrase Structure Parsing):

- Focus: **Phrases (NP, VP, PP)**
  Uses **Context-Free Grammar (CFG)**.

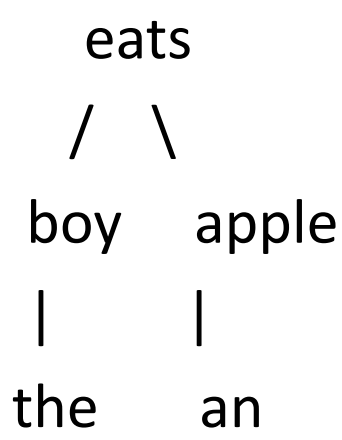
- Example tree:



2.Dependency Parsing:

Focus: word-to-word relations + Finds head–dependent links.

Example:



Approaches to Parsing

1. Top-Down Parsing

- Start from start symbol **S**
- Expand rules until you match the sentence
- Works like: "Predict first, match later“

2. Bottom-Up Parsing

- Start from words
- Combine them to form phrases
- Works like: "Build from words upward"

Duggirala Sri Ramya Krishna

3.Probabilistic Parsing (PCFG)

- Probabilistic parsing uses **probability** to choose the *best* parse tree when a sentence has multiple meanings.

Based on:

- PCFG – Probabilistic Context-Free Grammar
- Each grammar rule has a **probability**.

Why is it needed?

Some sentences can have more than one correct structure.

Example:

- I saw the man with a telescope.

Two possible meanings:

- I used the telescope
- The man had the telescope

Probabilistic parsing chooses the **more likely** meaning using rule probabilities

4. Chart Parsing (Dynamic Programming)

- Chart parsing is a parsing method that stores partial results so the parser does not repeat work.

It uses a chart (a table) to remember:

- which parts of the sentence are already recognized
- which grammar rules have been applied

Why is it important?

- Without chart parsing:
- Many combinations are rechecked
- Parsing becomes slow

Dynamic Parsing

It uses a **table (chart)** to store:

- Recognized words
- Partial phrases
- Completed constituents

When a phrase appears again:

- The parser **reads from the table** instead of recalculating.

This is the same principle used in:

- DP in algorithms
- Memoization

Sentence: **“the boy eats”**

Grammar:

S → NP VP

NP → Det N

VP → V

Applied rules:

Det → the

N → boy

V → eats

DP PARSING TABLE

Row	Span	Text Covered	Constituent Found	Explanation
1	0–1	the	**Det**	"the" matches rule **Det → the**
2	1–2	boy	**N**	"boy" matches rule **N → boy**
3	2–3	eats	**V**, **VP**	"eats" matches **V → eats**, and since **VP → V**, it also becomes a VP
4	0–2	the boy	**NP**	(0–1)=Det and (1–2)=N → satisfies rule **NP → Det N**
5	1–3	boy eats	—	(1–2)=N and (2–3)=V → no rule like **X → N V**, so nothing added
6	0–3	the boy eats	**S**	(0–2)=NP and (2–3)=VP → matches **S → NP VP**, so full sentence recognized

☆ Row 1: Span 0–1 → "the" → Det

- Look at first word: the
- Grammar says: Det → the

So the parser marks this word as a Determiner

☆ Row 2: Span 1–2 → "boy" → N

- Next word: boy
- Grammar says: N → boy

So parser identifies boy as a Noun

☆ Row 3: Span 2–3 → "eats" → V and VP

- Word: eats
- Matches: V → eats → It is a verb
- Grammar also says: VP → V
  So if something is a V, it is automatically a VP

Therefore “eats” is both V and VP

☆ Row 4: Span 0–2 → "the boy" → NP

- DP parser now tries to combine previous results:
  - (0–1) = Det
  - (1–2) = N
- Grammar rule: NP → Det N

So the boy is recognized as a Noun Phrase (NP)

☆ Row 5: Span 1–3 → "boy eats" → No constituent

- Combines:
  - (1–2) = N
  - (2–3) = V
- No rule in grammar like X → N V

So parser does NOT add anything

☆ Row 6: Span 0–3 → "the boy eats" → S

- Now combine:
  - (0–2) = NP
  - (2–3) = VP
- Grammar rule: S → NP VP

Shallow Parsing (Chunking)

The process of identifying **chunks** (small meaningful groups of words) in a sentence without building a full parse tree.

- It does **not** analyze full sentence structure like S → NP VP.
- Instead, it finds:
- Noun Phrases (NP)
- Verb Phrases (VP)
- Prepositional Phrases (PP)

✔ Also called:

- **Chunking**
- **Partial parsing**
- **Lightweight parsing**

"Shallow parsing helps find useful patterns without full grammar analysis".

Example of Shallow Parsing

Sentence: The quick brown fox jumps over the lazy dog.

Chunks identified:

- [NP The quick brown fox]
- [VP jumps]
- [PP over]
- [NP the lazy dog]

No full parse tree — only **phrase boundaries**.

What Shallow Parser Produces?

✔ **Noun Chunks (NP)**

- Group of words around a noun
  Example:
  the big dog

✔ **Verb Chunks (VP)**

- Group of words around a verb
  Example:
  is running

✔ **Prepositional Chunks (PP)**

- Preposition + noun chunk
  Example:
  in the park

Chunking Rules

Rule	Simple Meaning	Example
*NP → DT JJ NN**	A noun phrase may start with a determiner, have adjectives, and end with a noun	*the tall brown dog*
VP → AUX V	A verb phrase may contain helping verbs	*is eating*
PP → IN NP	A preposition always leads to a noun phrase	*on the table*
NP → PRP	Pronouns form noun phrases	*she*, *they*

Chunking Table Example

Span of Words	POS Pattern	Chunk Type	Explanation
The quick brown fox	DT JJ JJ NN	**NP (Noun Phrase)**	Matches rule: Determiner + adjectives + noun
jumps	VBZ	**VP (Verb Phrase)**	Verb alone forms a VP
over	IN	**PP (Prepositional Phrase)**	Single preposition becomes PP
the lazy dog	DT JJ NN	**NP (Noun Phrase)**	Determiner + adjective + noun

Example

We will parse the sentence:

I fish

- This sentence is ambiguous:
- Meaning 1 → “I catch fish”
- (fish = noun)
- Meaning 2 → “I fish”
- (fish = verb → “I am fishing”)
- A PCFG will choose the more likely meaning.

Grammar with Very Small Probabilities

Rule	Meaning	Probability
S → NP VP	sentence = noun phrase + verb phrase	**1.0**
NP → Pronoun	noun phrase is pronoun	**1.0**
VP → V	VP is a verb	**0.3**
VP → V NP	VP is verb + object	**0.7**
Pronoun → I	pronoun rule	**1.0**
V → fish	fish as **verb**	**0.6**
N → fish	fish as **noun**	**0.4**

How the above table obtained

where does the probability table come from?

- 👉 PCFG probabilities are NOT guessed.
- 👉 They are learned from data (Treebank).
- 👉 Small probabilities simply mean “this rule happens less often” in real language.

Example (imaginary small treebank):

- The fish swims
- Fish live in water
- She eats fish
- I fish
- We fish

Let's focus on the word "fish".

Use of "fish"	Count
fish as VERB	2 times
fish as NOUN	3 times

- Convert counts into probabilities:
- We use this formula : P(rule)=count of rule/total count
- Total occurrences = 5

Grammar Rule	Probability
V → fish	2/5=0.4
N → fish	3/5=0.6

Parse 1 (Verb-only): “I fish.”
(fish = verb → I am fishing)

Rule	Probability
S → NP VP	1.0
NP → Pronoun	1.0
Pronoun → I	1.0
VP → V	0.3
V → fish	0.4

$$P(\text{Parse 1}) = 1.0 \times 1.0 \times 1.0 \times 0.3 \times 0.4 = 0.12$$

Parse 2 (Verb + Object): “I catch fish.”
(fish = noun, VP = V NP)

Rule	Probability	Rule
S → NP VP	1.0	S → NP VP
NP → Pronoun	1.0	NP → Pronoun
Pronoun → I	1.0	Pronoun → I
VP → V NP	0.7	VP → V NP
V → fish	0.4	V → fish
NP → N	1.0	NP → N
N → fish	0.6	N → fish

- P(Parse 2)=1.0×1.0×1.0×0.7×0.4×1.0×0.6=0.168

Final Decision

Meaning	Probability
“I catch fish”	**0.168**
“I fish (I am fishing)”	0.12

✔ **0.168 is higher**

- So PCFG chooses:
- **“I catch fish.”**

CYK Algorithm

Sentence : “the fish eats”

Rule no	Grammar Rule
1	S → NP VP
2	NP → Det N
3	VP → V
4	Det → the
5	Rule No
6	V → eats

length **n = 3**

Position	Word
1	the
2	fish
3	eats

Algorithm

Step 1: Create an Empty CYK Table

- Rows = start position (i)
  Columns = substring length (l)

Start ↓ / Length →	1	2	3
1			
2			
3			

- We will fill this table step by step:

Step 2: Fill Length = 1 (Lexical Rules)

- Use rules of form A → word

Start	Word	Non-terminal
1	the	Det
2	fish	N
3	eats	V

- Updated CYK Table

Start ↓ / Length →	1	2	3
1	**Det**		
2	**N**		
3	**V**		

Step 3: Fill Length = 2 (Two-Word Phrases)

Cell (1,2): words the fish

- Split:
- (1,1) + (2,1)
- Det + N
- Grammar check:
- NP → Det N ✔

Cell (2,2): words fish eats

- Split:
- (2,1) + (3,1)
- N + V
- Grammar check:
- No rule N V ✘

Start ↓ / Length →	1	2	3
1	Det	**NP**	
2	N	–	
3	V		

Step 4: Fill Length = 3 (Full Sentence)

Cell (1,3): words the fish eats

- Possible splits:
- **Split 1:**
- (1,1) + (2,2)
- Det + –
- ✗ No rule
- **Split 2:**
- (1,2) + (3,1)
- NP + V
- Grammar:
- VP → V ✔
- S → NP VP ✔

Start ↓ / Length →	1	2	3
1	Det	NP	**S**
2	N	–	
3	V		

- Acceptance Condition (Very Important)
- ✔ If S appears in (1,n)
  → Sentence is ACCEPTED
- ✔ Here, S is in (1,3) → accepted

Splitting table

Word group	Left part	Right part	Why this split
the	–	–	single word → no split
fish	–	–	single word → no split
eats	–	–	single word → no split
the fish	the	fish	only way to cut 2 words
fish eats	fish	eats	only way to cut 2 words
the fish eats	the	fish eats	first possible cut
	the fish	eats	second possible cut

Cyk cells calculation

CYK cell	Words covered	Left cell	Right cell
(1,1)	the	–	–
(2,1)	fish	–	–
(3,1)	eats	–	–
(1,2)	the fish	(1,1)	(2,1)
(2,2)	fish eats	(2,1)	(3,1)
(1,3)	the fish eats	(1,1)	(2,2)
		(1,2)	(3,1)

Probabilistic CYK

Probabilistic CYK (PCYK) works exactly like CYK, but:

- Every grammar rule has a probability
- PCYK chooses the most probable parse
- Same splitting, extra probability calculation

S: the dogs run

1 2 3

PCFG with Different Probabilities

Grammar Rule	Probability
S → NP VP	1.0
NP → Det N	0.8
NP → N	0.2
VP → V	0.6
VP → V NP	0.4
Det → the	1.0
N → dogs	1.0
V → run	1.0

Step 1 – Length = 1 (Lexical Cells)

Using rules **A → word**

Cell	Word	Non-terminal	Probability
(1,1)	the	Det	1.0
(2,1)	dogs	N	1.0
(3,1)	run	V	1.0

Step 2 – Length = 2 (Splitting)

Cell (1,2): *the dogs:*

Left	Right	Rule	Probability
Det	N	NP → Det N	1.0 × 1.0 × 0.8 = **0.8**

(1,2) = NP(0.8)

Cell (2,2): *dogs run:*

Left	Right	Rule	Result
N	V	No rule	–

Step 3 – Length = 3

- Cell (1,3): *the dogs run*

k	Left Cell	Right Cell	Result
1	(1,1)=Det	(2,2)=–	✗
2	(1,2)=NP(0.8)	(3,1)=V(1.0)	✓

Rule	Probability
VP → V	0.6
S → NP VP	1.0

- VP = 1.0 × 0.6 = 0.6
- S = 0.8 × 0.6 × 1.0 = 0.48

- (1,3) = S(0.48)

Final PCYK Table

Start ↓ / Length →	1	2	3
1	Det	NP(0.8)	**S(0.48)**
2	N	–	
3	V		

What is a Probabilistic Lexicalized CFG?

- A PCFG in which each phrase is associated with a head word, and grammar rule probabilities depend on those words.
- Structure + Probability + Words
- Lexicalized = Head word is attached to each phrase

Ex: dogs run

- VP(run) → V(run)
- S(run) → NP(dogs) VP(run)

Why Probability + Lexicalization Together?

- Words strongly influence structure
- Some verbs prefer certain objects
- Probabilities help choose the **most natural parse**
- Ex:

eat → apple (high probability)

eat → idea (low probability)

- Ex :birds eat fish

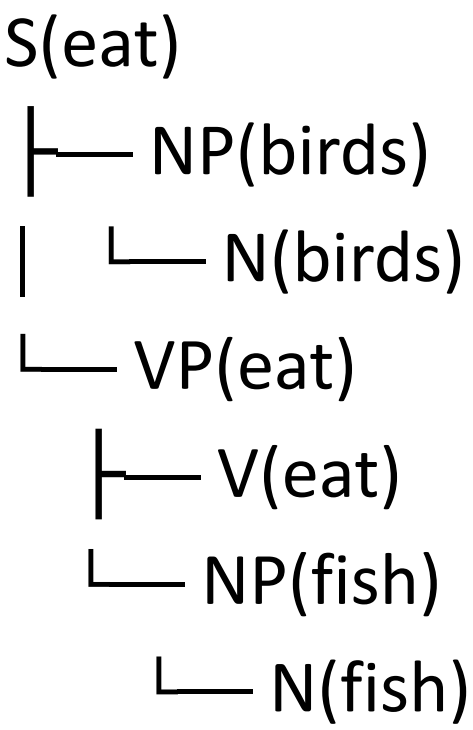
Level	Phrase / Rule	Lexicalized Form (with Head)	Probability	Explanation
Word	birds	N(birds)	1.0	“birds” is a noun
Word	eat	V(eat)	1.0	“eat” is a verb
Word	fish	N(fish)	1.0	“fish” is a noun
Phrase	NP	NP(birds) → N(birds)	0.9	Noun phrase headed by “birds”
Phrase	NP	NP(fish) → N(fish)	0.8	Noun phrase headed by “fish”
Phrase	VP	VP(eat) → V(eat) NP(fish)	0.85	Verb “eat” prefers object “fish”
Sentence	S	S(eat) → NP(birds) VP(eat)	1.0	Sentence headed by main verb “eat”

How to Read This Table

- Lexicalized means every phrase has a head word
- NP(birds), VP(eat), S(eat)
- Probabilities depend on words, not just symbols
- Rule probabilities reflect how natural the word combination is

- *eat* → *fish* has high probability
- Final parse probability is the product of all rule probabilities

Parse tree



What are Feature Structures?

A Feature Structure (FS) is a way to represent linguistic information using:

- Attribute – Value pairs

They describe properties of words or phrases such as:

- Number
- Gender
- Tense
- Person
- Case

Why Feature Structures are Needed

- CFG rules alone cannot capture agreement constraints.

- Example problem:

1.The boy eat apples ✗

2.The boy eats apples ✔

CFG allows both, but feature structures enforce agreement.

Word: “dogs”

- A feature structure represents information as **attribute–value pairs**.

Feature	Value
CAT	N (Noun)
NUM	plural
PERSON	third
TENSE	–

- This table is the **feature structure** of the word **dogs**.

Word: “eat”

Feature	Value
CAT	V (Verb)
NUM	plural
PERSON	third
TENSE	present

- English Subject–Verb Agreement

Subject	Verb form (present tense)
Singular noun	Verb is **singular**
Plural noun	Verb is **plural**

Feature Structure for a Phrase (NP)

- Phrase: “the dogs”

Feature	Value
CAT	NP
NUM	plural
PERSON	third

- The number (plural) comes from the head noun dogs.

Rule with feature variables:

- NP[NUM = X] → Det[NUM = X] N[NUM = X]

Meaning:

- NP, Det, and N **must have the same number**
- Feature **X** ensures agreement

The dogs eat

Word	NUM
the	plural
dogs	plural
eat	plural

- All feature structures are compatible
  - ✔ Sentence is **grammatically correct**

What is Unification?

Unification is the process of:

- Combining two feature structures into one, only if there is no conflict in feature values.

- ✔ Same feature + same value → OK
  ✘ Same feature + different value → FAIL

Example 1 – Unification SUCCESS (Correct Sentence)

S : The dogs eat

Feature Structures:

Word	CAT	NUM	PERSON
dogs	N	plural	third
eat	V	plural	third

- S → NP[NUM=X] VP[NUM=X]

Unified Feature Structure

Feature	Value
NUM	plural
PERSON	third

No conflict

- ✔ Unification succeeds
- ✔ Sentence is grammatical

Example 2 – Unification FAILURE (Incorrect Sentence)
The dogs eats

- Feature Structures

Word	CAT	NUM
dogs	N	plural
eats	V	singular

- conflict

Feature	NP	VP
NUM	plural	singular

- ✗ Values do not match
  ✗ **Unification fails**
  ✗ Sentence is **ungrammatical**